VIBE CODING
& AI TRAINING

A practical course catalogue, daily teaching plans, pricing strategy, and instructor preparation guide

Training setup: In-person classes in an air-conditioned room, with laptops provided by the instructor.

Recommended starting point: Launch the 10-day Vibe Coding course first. Add Android after you have tested the projects yourself. Introduce backend and agentic AI only after your own preparation.

Prepared as an actionable starting plan. Fees are suggested pricing in Pakistani rupees, not verified market averages. Adjust based on student demand, batch size, support, and local costs.

1. Executive summary

You already have laptops and an air-conditioned classroom. The best first product is a practical beginner course where students use AI coding tools to build and publish a website. This has a manageable scope and lets you develop teaching confidence before covering backend systems or autonomous AI agents.

Recommended course lineup

Course	Duration	Suggested fee / student	Launch priority
Vibe Coding: Websites & Web Apps	10 days 2 hrs/day	Rs. 4,999–7,999	**Start here**
AI Android App Development	12–15 days 2 hrs/day	Rs. 6,999–9,999	After practice
Agentic AI & Automation	12 days 2 hrs/day	Rs. 8,999–12,999	Later
AI Full-Stack Builder Bootcamp	20 days 2 hrs/day	Rs. 14,999–19,999	Advanced / later

Important: The first course can stay frontend-focused. Do not promise that complete beginners will master backend development or agentic AI in a short class. Students should leave with working projects and a realistic understanding of what they can and cannot yet build.

2. How to run every class

Use the same two-hour routine each day. Predictable structure helps students practise instead of passively watching.

Time	Activity	What happens
15 min	**Concept**	Explain the idea, vocabulary, and today's goal.
25 min	**Live demo**	Build one feature and narrate the prompts and decisions.
55 min	**Hands-on build**	Students recreate the feature and customise it.
15 min	**Challenge**	Give a small task they solve with less help.
10 min	**Review**	Check progress, answer questions, and set practice.

Instructor rule: Never rely on an AI-generated result without checking it. Demonstrate how to read errors, inspect changes, test key features, and recover when something fails.

3. Course 1 — Vibe Coding: Websites & Web Apps

Duration: 10 days · Daily class: 2 hours · Suggested fee: Rs. 4,999–7,999 · Level: Beginner

Course outcome: Each student builds a responsive business website, adds a contact/lead form, uses GitHub for source control, and publishes a live link. Keep the project frontend-first; a real database and authentication can be optional extension topics.

Day	Topic	Teach	Deliverable / check
1	**Vibe coding and setup**	Explain what AI coding tools can/cannot do; project briefs; installing browser/editor/tools; create a folder and start a project.	Personal landing page opens locally; student can explain the prompt and identify project files.
2	**Prompting and requirements**	Teach role/context, objective, audience, constraints, examples, acceptance criteria, and iterative prompting.	Business landing page with clear sections and a consistent visual direction.
3	**Web fundamentals**	Explain HTML structure, CSS styling, JavaScript behaviour, components, files, and browser inspection.	Responsive multi-section website; student can locate the relevant HTML/CSS/JS or component files.
4	**UI/UX and responsive design**	Typography, spacing, hierarchy, color, navigation, mobile breakpoints, accessibility basics, and image optimisation.	Polished homepage that works on mobile and desktop.
5	**Interactions and forms**	Buttons, form inputs, validation, loading/error states, client-side state, and clear success messages.	Lead-capture form with validation. Explain that a frontend-only form does not automatically store or send leads.
6	**APIs and JSON**	Explain requests/responses, JSON, GET requests, async loading, error states, and API keys. Use a safe public demo API.	A page that fetches and displays external data. Keep secret keys out of browser code.
7	**Data and backend concepts**	Explain frontend vs backend, database vs API, CRUD, and why secure storage/auth require server-side work. Use a guided demo or managed service only if tested.	Small data-management prototype. Label any mock/local data clearly; do not pretend it is a production backend.
8	**Debugging and Git**	Read console errors, reproduce a bug, ask AI for a minimal fix, review diffs, commit changes, and push to GitHub.	Fix a deliberately broken feature and make a repository commit.
9	**Testing and deployment**	Test forms and links, mobile layouts, accessibility basics, broken images, browser console, and deploy to a static hosting provider.	Published website with a working public URL.
10	**Capstone and presentation**	Students finish a business site, test it using a checklist, explain design choices, show source code, and present limitations.	Complete website, GitHub repository, README, and short presentation.

Suggested capstone options

- Local business website: home, services, about, contact, mobile layout, and a clear call to action.
- Personal portfolio: projects, skills, contact section, and live deployment.
- Restaurant site: menu, hours, location map embed, and an enquiry form.

Assessment checklist

- Site opens from a public link and has no obvious broken sections.
- Layout works at phone and desktop widths.
- Navigation, buttons, and form validation behave as expected.
- Student can explain the project structure and fix a small issue without blindly regenerating everything.
- README explains how to run the project and identifies limitations.

4. Course 2 — AI Android App Development

Duration: 12–15 days · Daily class: 2 hours · Suggested fee: Rs. 6,999–9,999 · Level: Beginner to intermediate

Prerequisite: Basic computer confidence. Students should be comfortable creating folders, installing software, and following technical steps. Test Android Studio, SDK downloads, emulator performance, and physical-device debugging before enrolling a class.

Course outcome: A working Android app, source code, tested APK or other suitable build artifact, and an understanding of the release process. Do not guarantee Play Store approval.

Day	Topic	Teach	Deliverable / check
1	**Android setup**	Explain Android apps, SDK, Android Studio, project templates, emulator, and real-device options. Confirm every laptop can build the starter app.	Starter project runs.
2	**AI-assisted planning**	Write app requirements, screen list, navigation map, data needs, acceptance criteria, and a stepwise prompt. Introduce Google AI Studio only as an optional prototyping route.	App plan and first prototype.
3	**Kotlin essentials**	Variables, types, functions, conditions, lists, nullability basics, and reading AI-generated Kotlin.	Small functions and simple app logic.
4	**Jetpack Compose UI**	Composable functions, layouts, modifiers, text, buttons, lists, themes, and Material components.	Styled home screen.
5	**Navigation and state**	Multiple screens, navigation, user input, UI state, validation, and empty/loading states.	Multi-screen task or notes app.
6	**Local persistence**	Data models, simple local storage options, and saving/retrieving app data. Explain trade-offs between preferences and structured local databases.	Data remains after restarting the app.
7	**AI agent mode and code review**	Use Android Studio's available AI assistance to plan a task, inspect proposed changes, edit multiple files, and review diffs. Confirm current feature availability in the installed version.	One new feature with reviewed changes.
8	**Networking and APIs**	HTTP basics, JSON models, asynchronous work, loading/error states, network permissions, and handling failed requests.	App fetches data from a test API.
9	**Firebase/cloud basics**	Explain authentication, database rules, environment configuration, and why client apps must not contain privileged secrets. Use a pre-tested starter configuration.	Basic sign-in/cloud-data demo, or a clearly labelled mock if setup fails.
10	**Testing and debugging**	Logcat, emulator vs device, permissions, crash reproduction, basic test cases, and regression checks.	Fixed issues and a test checklist.
11	**Build and release basics**	App icon, versioning, signing, APK vs AAB, privacy/data disclosures, and Play Console overview. Signing keys must be stored securely.	Release build generated; no publishing promise.
12	**Capstone**	Complete a small app, test on emulator/device, review code, document setup, and present the result.	Installable app, source code, and demo.

Optional days 13–15

- Day 13: Add a feature and practise refactoring with AI while reviewing every change.
- Day 14: UI polish, accessibility, offline/error states, and a fuller test pass.
- Day 15: Release checklist, app listing assets, demo day, and individual project reviews.

Suggested capstones

- Task manager with local persistence.
- Student timetable or study planner.
- Simple inventory list for a small shop.
- Weather or public-data app using a safe API.

Important teaching boundary

Do not introduce payment processing, sensitive personal data, production authentication, or complex cloud rules until you can configure and debug them yourself. A generated UI is not proof that the app is secure or production-ready.

5. Course 3 — Agentic AI & Automation

Duration: 12 days · Daily class: 2 hours · Suggested fee: Rs. 8,999–12,999 · Level: Intermediate

Do not launch this course until you have built the capstone yourself, can explain tool calling and agent loops, and have tested failure cases. This course requires API/backend basics. Students should know at least basic JavaScript or Python, or complete a prerequisite module.

Day	Topic	Teach	Deliverable / check
1	**LLMs, chatbots, and agents**	Explain model input/output, system instructions, ordinary chat vs tool-using agents, goals, tool permissions, and where autonomy can fail.	Written agent design for one bounded problem.
2	**Prompts and structured outputs**	Context, output schemas, JSON validation, examples, and handling malformed model responses.	Assistant returns validated structured data.
3	**API and backend foundations**	HTTP requests, environment variables, server-side API calls, key management, error responses, and rate limits.	Small backend endpoint that calls an AI model.
4	**Tool/function calling**	Tool schemas, arguments, validation, allowlisted functions, and separating model suggestions from actual execution.	Agent uses a calculator or controlled local lookup.
5	**Agent execution loop**	Plan, choose tool, validate, execute, observe, continue/stop, maximum steps, and timeout handling.	Bounded multi-step workflow with a stop condition.
6	**Knowledge retrieval and memory**	Difference between conversation history and long-term memory; document chunks, retrieval, relevance, and source-grounded answers.	Document Q&A; demo using a small approved dataset.
7	**External APIs and webhooks**	API authentication, webhooks, mock CRM records, test environments, and handling duplicate requests.	Agent reads/writes mock CRM data.
8	**Workflow automation**	Triggers, filters, branches, schedules, and actions in an automation platform or simple code workflow.	Lead classification and routing workflow.
9	**Multi-agent concepts**	Specialized roles, orchestration, shared state, coordination costs, and when a single agent is simpler.	Research-and-summary workflow with clear handoff.
10	**Evaluation and reliability**	Test cases, traces/logs, retries, timeouts, model variability, cost tracking, and human review.	Evaluation set with recorded results.
11	**Security and approvals**	Prompt injection, data minimization, permissions, secret handling, allowlists, human approval for external actions, and audit logs.	Safe workflow that drafts but does not send without approval.
12	**Capstone and demo**	Run end-to-end tests, document limitations, deploy only if prepared, present workflow and failure handling.	Working controlled AI workflow and technical handover.

Capstone specification

Build a lead assistant that accepts sample leads, classifies intent, summarises the request, drafts a response, and routes it to a mock CRM. Require human approval before sending messages, editing real records, or performing irreversible actions.

Instructor readiness checklist

- Can explain each tool schema and validate its inputs.
- Can show the execution trace and explain why the agent stopped.
- Can handle missing credentials, invalid outputs, timeouts, and failed tools.
- Can demonstrate prompt-injection tests and permission boundaries.
- Can estimate API costs and explain that model outputs may be incorrect.

6. Course 4 — AI Full-Stack Builder Bootcamp

Duration: 20 days · Daily class: 2 hours · Suggested fee: Rs. 14,999–19,999 · Level: Advanced

Run this only after you can build and troubleshoot a full-stack project yourself. Students should have frontend basics; this is not a first-ever coding course.

Day	Topic	Teach	Deliverable
1	**Product scope and architecture**	Requirements, users, feature prioritisation, data flow, and acceptance criteria.	Project spec and architecture sketch.
2	**Project structure and Git**	Repository, environment setup, commits, branches, and safe AI coding workflow.	Clean repo with first commit.
3	**Frontend components**	Component boundaries, reusable UI, forms, and layout.	Working frontend skeleton.
4	**Responsive UI and state**	Responsive layouts, navigation, client state, and loading/error states.	Multi-page interface.
5	**Language and validation**	JavaScript/TypeScript essentials, types, validation, and code readability.	Validated forms and data models.
6	**Backend and HTTP**	Routes, REST concepts, request/response handling, and status codes.	API service.
7	**Database and CRUD**	Tables/collections, relationships, constraints, migrations or schema changes.	Persistent CRUD operations.
8	**Authentication and roles**	Sessions/tokens at a high level, access control, protected routes, and secure configuration.	Protected user data.
9	**Integrations and files**	External APIs, uploads, file limits, and error handling.	Integrated feature.
10	**Backend testing**	API tests, logs, input validation, and consistent errors.	Tested API.
11	**AI integration**	Server-side model calls, prompts, structured outputs, and rate limits.	AI feature.
12	**Validate AI outputs**	Schema validation, refusals, fallback paths, and safe presentation of results.	Robust AI response handling.
13	**Tool calling**	Tool definitions, argument checks, allowlisted actions, and step limits.	Tool-using assistant.
14	**Retrieval and knowledge**	Document ingestion, chunking concepts, retrieval quality, and citations to source material.	Knowledge assistant.
15	**Security review**	Secrets, permissions, input validation, dependency checks, privacy, and prompt injection.	Security checklist and fixes.
16	**Testing strategy**	Unit, integration, end-to-end smoke tests, and regression testing.	Repeatable test suite.
17	**Deployment**	Build settings, environment variables, hosting, database configuration, and migrations.	Staging deployment.
18	**Operations and cost**	Monitoring, backups, API usage, limits, recovery, and basic performance.	Operations checklist.
19	**Polish and documentation**	UX fixes, accessibility, README, setup instructions, and known limitations.	Release candidate.
20	**Demo and review**	Present product, test key flows, explain architecture, and receive feedback.	Portfolio-ready capstone.

Suggested capstones

- Appointment-booking application with admin dashboard.
- Inventory tracker with role-based access.
- AI customer-support assistant grounded in approved documents.
- Lead management CRM with AI-generated draft replies and human approval.

7. Instructor preparation plan: 14 days

Because your backend and agentic AI experience is limited, prepare before selling advanced material. This schedule gets the first website course ready; it does not certify you to teach advanced backend or agentic AI yet.

Day	Focus	Proof you are ready
1	**Prompting and setup**	Build a personal portfolio with an AI coding tool; understand where files live.
2	**Landing pages**	Build a business landing page from a written brief.
3	**Responsive design**	Test phone and desktop sizes; fix layout issues.
4	**Frontend fundamentals**	Learn enough HTML, CSS, JavaScript and component concepts to explain generated code.
5	**Debugging**	Break a feature intentionally, inspect errors, and fix it without replacing the entire project.
6	**Forms**	Build and validate a form; explain what happens if it is frontend-only.
7	**APIs**	Call a safe public API; understand requests, JSON, loading and failure states.
8	**Git and GitHub**	Commit, review changes, push, and restore an earlier working version.
9	**Deployment**	Publish a static site and test its public link.
10	**Build from scratch**	Create a complete business website without copying a tutorial step-by-step.
11	**Second project**	Build a restaurant, portfolio, or local-service site from a new brief.
12	**Mock teaching**	Deliver a 30–45 minute lesson aloud; note where explanations are unclear.
13	**Prepare course assets**	Create slides, prompts, starter files, exercises, solution notes, and a rubric.
14	**Full rehearsal**	Run a two-hour mock class from setup through review and homework.

After the first course, prepare the Android course by building at least two apps yourself, including one using local storage and one that calls an API. Prepare the agentic AI course only after you can independently build, test, and explain a tool-using workflow.

8. Classroom and equipment checklist

- Install/test browser, code editor, Git, and course-specific tools on every laptop.
- For Android: test Android Studio, SDKs, emulator performance, storage space, and USB/device debugging.
- Check Wi-Fi capacity, power outlets, chargers, extension boards, and backup internet if possible.
- Use a sample project and backup recording/screenshots if a service outage interrupts the lesson.
- Ask students to use their own AI accounts when possible; review current free-tier limits before class.
- Do not share API keys or passwords in class slides, public repositories, or frontend code.
- Keep starter projects, lesson notes, assignments, and solutions organised by day.
- Set classroom rules for account privacy, respectful collaboration, attendance, and project attribution.

9. Pricing, revenue, and launch plan

Suggested fees

Course	Suggested launch fee	Potential later fee
Vibe Coding, 10 days	Rs. 4,999	Rs. 6,999–7,999
Android, 12–15 days	Rs. 6,999	Rs. 8,999–9,999
Agentic AI, 12 days	Rs. 8,999	Rs. 10,999–12,999
Full-stack, 20 days	Rs. 14,999	Rs. 17,999–19,999

Illustrative batch revenue

Students enrolled	Fee per student	Gross revenue
5	Rs. 4,999	**Rs. 24,995**
10	Rs. 4,999	**Rs. 49,990**
15	Rs. 4,999	**Rs. 74,985**
20	Rs. 4,999	**Rs. 99,980**

Gross revenue is not profit. Even if the room and laptops are already available, account for instructor time, internet, electricity, maintenance, marketing, software/API usage, taxes, and the opportunity cost of your time. Confirm that your seat count is manageable for hands-on support.

Suggested launch sequence

- Week 1: Prepare and rehearse the first website course; create two demonstration projects.
- Week 2: Promote the course with real demos and a clear syllabus; run a free introductory workshop.
- Week 3: Enrol a small pilot group and confirm students can access required tools.
- Week 4: Deliver the course, collect feedback, and request permission before sharing student projects or testimonials.
- After the pilot: Improve lessons, then prepare and launch Android as a separate batch.
- Only launch agentic AI and full-stack courses when your own projects and troubleshooting guides are ready.

10. Student assessment and completion standards

Use practical checks rather than attendance alone. A certificate should mean the student completed the stated work; it should not imply professional mastery.

Assessment area	What to check	Suggested weight
Working functionality	Core features work; no major broken navigation or unhandled errors.	30%
Design / usability	Clear layout, mobile responsiveness, readable text, sensible interaction.	20%
Code understanding	Student can locate key files, explain an AI-generated change, and debug a small issue.	20%
Testing and safety	Student tests common cases and understands important limitations and data handling.	15%
Presentation and documentation	Student demonstrates the project and supplies basic setup instructions.	15%

11. What not to promise

- Do not promise guaranteed income, freelance clients, jobs, app-store approval, or production security.
- Do not call a mockup or locally stored prototype a complete production application.
- Do not tell beginners they can build anything without understanding code; AI output must be reviewed and tested.
- Do not teach advanced backend or agentic AI before you can reproduce the complete demo and troubleshoot common failures.
- Do not require students to share passwords, private keys, personal customer records, or sensitive business data.

12. Tool stack and official learning references

Use the tools that fit your chosen project; you do not need to teach every platform at once. Check current features, account requirements, pricing, and free-tier limits before each batch.

Resource	Official link	Use it for
Google AI Studio	https://aistudio.google.com/	Prompt-driven prototyping and AI experiments.
Android Studio	https://developer.android.com/studio	Official Android IDE downloads and documentation.
Android Developers	https://developer.android.com/	Kotlin, Jetpack Compose, testing, and publishing guidance.
GitHub	https://github.com/	Repositories and version control.
MDN Web Docs	https://developer.mozilla.org/	HTML, CSS, JavaScript, and browser fundamentals.
Git documentation	https://git-scm.com/doc	Git concepts and commands.

13. Final recommendation

Start with one small, high-quality Vibe Coding cohort. Teach websites, frontend concepts, debugging, Git, and deployment. Build the entire course yourself first. Once you can confidently reproduce the work and answer students' questions, add Android. Treat backend and agentic AI as later specialisations that require additional preparation.

This plan is a teaching and business framework. Validate every lesson on the exact laptops and accounts you will use in class, and update tool instructions as platforms change.